5.5 Using Array-Based Sequences

5.5.1 Storing High Scores for a Game

The first application we study is storing a sequence of high score entries for a video game. This is representative of many applications in which a sequence of objects must be stored. We could just as easily have chosen to store records for patients in a hospital or the names of players on a football team. Nevertheless, let us focus on storing high score entries, which is a simple application that is already rich enough to present some important data-structuring concepts.

To begin, we consider what information to include in an object representing a high score entry. Obviously, one component to include is an integer representing the score itself, which we identify as _score. Another useful thing to include is the name of the person earning this score, which we identify as _name. We could go on from here, adding fields representing the date the score was earned or game statistics that led to that score. However, we omit such details to keep our example simple. A Python class, GameEntry, representing a game entry, is given in Code Fragment 5.7.

```
1  class GameEntry:
2    """Represents one entry of a list of high scores."""
3
4    def __init__(self, name, score):
5      self._name = name
6      self._score = score
7
8    def get_name(self):
9      return self._name
10
11   def get_score(self):
12     return self._score
13
14   def __str__(self):
15     return '({0}, {1})'.format(self._name, self._score) # e.g., '(Bob, 98)'
```

Code Fragment 5.7: Python code for a simple GameEntry class. We include methods for returning the name and score for a game entry object, as well as a method for returning a string representation of this entry.

A Class for High Scores

To maintain a sequence of high scores, we develop a class named Scoreboard. A scoreboard is limited to a certain number of high scores that can be saved; once that limit is reached, a new score only qualifies for the scoreboard if it is strictly higher than the lowest "high score" on the board. The length of the desired scoreboard may depend on the game, perhaps 10, 50, or 500. Since that limit may vary depending on the game, we allow it to be specified as a parameter to our Scoreboard constructor.

Internally, we will use a Python list named _board in order to manage the GameEntry instances that represent the high scores. Since we expect the scoreboard to eventually reach full capacity, we initialize the list to be large enough to hold the maximum number of scores, but we initially set all entries to None. By allocating the list with maximum capacity initially, it never needs to be resized. As entries are added, we will maintain them from highest to lowest score, starting at index 0 of the list. We illustrate a typical state of the data structure in Figure 5.18.

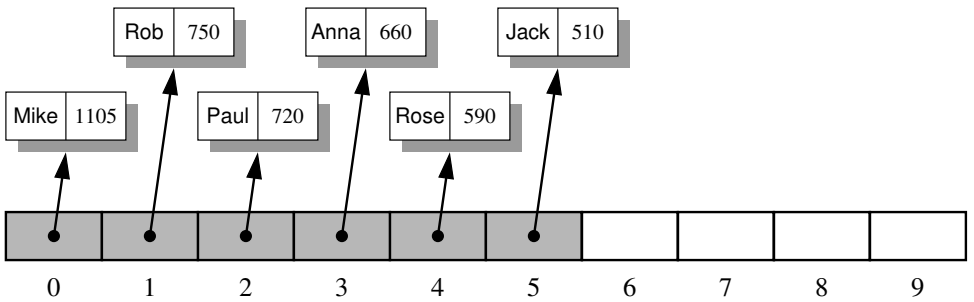


Figure 5.18: An illustration of an ordered list of length ten, storing references to six GameEntry objects in the cells from index 0 to 5, with the rest being None.

A complete Python implementation of the Scoreboard class is given in Code Fragment 5.8. The constructor is rather simple. The command

self._board = [**None**] * capacity

creates a list with the desired length, yet all entries equal to None. We maintain an additional instance variable, _n, that represents the number of actual entries currently in our table. For convenience, our class supports the __getitem__ method to retrieve an entry at a given index with a syntax board[i] (or None if no such entry exists), and we support a simple __str__ method that returns a string representation of the entire scoreboard, with one entry per line.

```
1  class Scoreboard:
2    """Fixed-length sequence of high scores in nondecreasing order."""
3
4    def __init__(self, capacity=10):
5      """Initialize scoreboard with given maximum capacity.
6
7      All entries are initially None.
8      """
9      self._board = [None] * capacity          # reserve space for future scores
10     self._n = 0                              # number of actual entries
11
12   def __getitem__(self, k):
13     """Return entry at index k."""
14     return self._board[k]
15
16   def __str__(self):
17     """Return string representation of the high score list."""
18     return '\n'.join(str(self._board[j]) for j in range(self._n))
19
20   def add(self, entry):
21     """Consider adding entry to high scores."""
22     score = entry.get_score( )
23
24     # Does new entry qualify as a high score?
25     # answer is yes if board not full or score is higher than last entry
26     good = self._n < len(self._board) or score > self._board[-1].get_score( )
27
28     if good:
29       if self._n < len(self._board):         # no score drops from list
30         self._n += 1                         # so overall number increases
31
32       # shift lower scores rightward to make room for new entry
33       j = self._n - 1
34       while j > 0 and self._board[j-1].get_score( ) < score:
35         self._board[j] = self._board[j-1]    # shift entry from j-1 to j
36         j -= 1                               # and decrement j
37       self._board[j] = entry                 # when done, add new entry
```

Code Fragment 5.8: Python code for a Scoreboard class that maintains an ordered series of scores as GameEntry objects.

Adding an Entry

The most interesting method of the Scoreboard class is add, which is responsible for considering the addition of a new entry to the scoreboard. Keep in mind that every entry will not necessarily qualify as a high score. If the board is not yet full, any new entry will be retained. Once the board is full, a new entry is only retained if it is strictly better than one of the other scores, in particular, the last entry of the scoreboard, which is the lowest of the high scores.

When a new score is considered, we begin by determining whether it qualifies as a high score. If so, we increase the count of active scores, _n, unless the board is already at full capacity. In that case, adding a new high score causes some other entry to be dropped from the scoreboard, so the overall number of entries remains the same.

To correctly place a new entry within the list, the final task is to shift any inferior scores one spot lower (with the least score being dropped entirely when the scoreboard is full). This process is quite similar to the implementation of the insert method of the list class, as described on pages 204–205. In the context of our scoreboard, there is no need to shift any None references that remain near the end of the array, so the process can proceed as diagrammed in Figure 5.19.

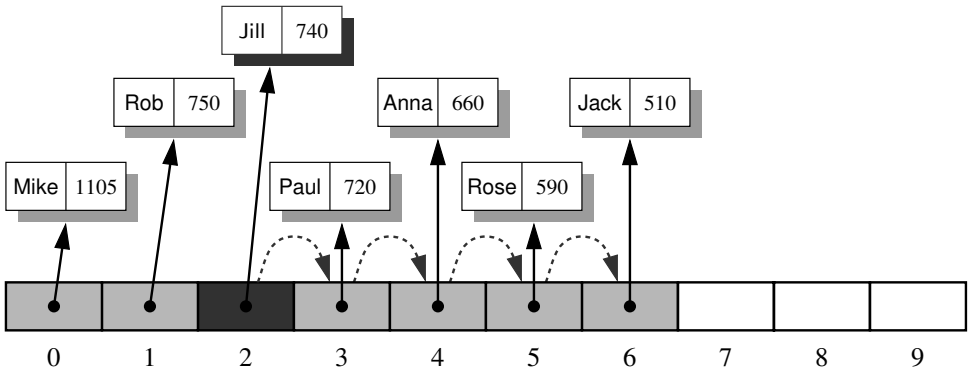


Figure 5.19: Adding a new GameEntry for Jill to the scoreboard. In order to make room for the new reference, we have to shift the references for game entries with smaller scores than the new one to the right by one cell. Then we can insert the new entry with index 2.

To implement the final stage, we begin by considering index j = self._n − 1, which is the index at which the last GameEntry instance will reside, after completing the operation. Either j is the correct index for the newest entry, or one or more immediately before it will have lesser scores. The while loop at line 34 checks the compound condition, shifting references rightward and decrementing j, as long as there is another entry at index $j-1$ with a score less than the new score.

5.5.2 Sorting a Sequence

In the previous subsection, we considered an application for which we added an object to a sequence at a given position while shifting other elements so as to keep the previous order intact. In this section, we use a similar technique to solve the ***sorting*** problem, that is, starting with an unordered sequence of elements and rearranging them into nondecreasing order.

The Insertion-Sort Algorithm

We study several sorting algorithms in this book, most of which are described in Chapter 12. As a warm-up, in this section we describe a nice, simple sorting algorithm known as ***insertion-sort***. The algorithm proceeds as follows for an array-based sequence. We start with the first element in the array. One element by itself is already sorted. Then we consider the next element in the array. If it is smaller than the first, we swap them. Next we consider the third element in the array. We swap it leftward until it is in its proper order with the first two elements. We then consider the fourth element, and swap it leftward until it is in the proper order with the first three. We continue in this manner with the fifth element, the sixth, and so on, until the whole array is sorted. We can express the insertion-sort algorithm in pseudo-code, as shown in Code Fragment 5.9.

Algorithm InsertionSort(A):
Input: An array A of n comparable elements
Output: The array A with elements rearranged in nondecreasing order
for k from 1 to n − 1 **do**
Insert A[k] at its proper location within A[0], A[1], ..., A[k].

Code Fragment 5.9: High-level description of the insertion-sort algorithm.

This is a simple, high-level description of insertion-sort. If we look back to Code Fragment 5.8 of Section 5.5.1, we see that the task of inserting a new entry into the list of high scores is almost identical to the task of inserting a newly considered element in insertion-sort (except that game scores were ordered from high to low). We provide a Python implementation of insertion-sort in Code Fragment 5.10, using an outer loop to consider each element in turn, and an inner loop that moves a newly considered element to its proper location relative to the (sorted) subarray of elements that are to its left. We illustrate an example run of the insertion-sort algorithm in Figure 5.20.

The nested loops of insertion-sort lead to an $O(n^2)$ running time in the worst case. The most work is done if the array is initially in reverse order. On the other hand, if the initial array is nearly sorted or perfectly sorted, insertion-sort runs in $O(n)$ time because there are few or no iterations of the inner loop.

```
1  def insertion_sort(A):
2    """Sort list of comparable elements into nondecreasing order."""
3    for k in range(1, len(A)):                # from 1 to n-1
4      cur = A[k]                              # current element to be inserted
5      j = k                                   # find correct index j for current
6      while j > 0 and A[j−1] > cur:           # element A[j-1] must be after current
7        A[j] = A[j−1]
8        j −= 1
9      A[j] = cur                              # cur is now in the right place
```

Code Fragment 5.10: Python code for performing insertion-sort on a list.

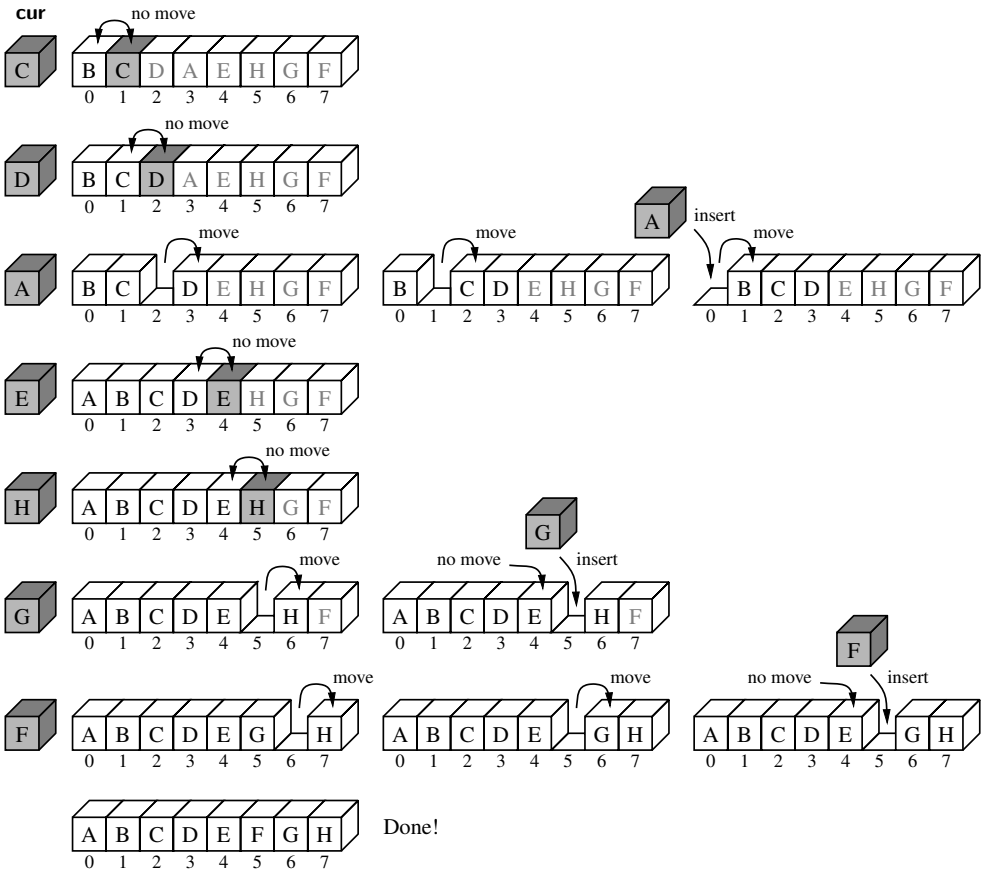


Figure 5.20: Execution of the insertion-sort algorithm on an array of eight characters. Each row corresponds to an iteration of the outer loop, and each copy of the sequence in a row corresponds to an iteration of the inner loop. The current element that is being inserted is highlighted in the array, and shown as the cur value.

5.5.3 Simple Cryptography

An interesting application of strings and lists is ***cryptography***, the science of secret messages and their applications. This field studies ways of performing ***encryption***, which takes a message, called the ***plaintext***, and converts it into a scrambled message, called the ***ciphertext***. Likewise, cryptography also studies corresponding ways of performing ***decryption***, which takes a ciphertext and turns it back into its original plaintext.

Arguably the earliest encryption scheme is the ***Caesar cipher***, which is named after Julius Caesar, who used this scheme to protect important military messages. (All of Caesar's messages were written in Latin, of course, which already makes them unreadable for most of us!) The Caesar cipher is a simple way to obscure a message written in a language that forms words with an alphabet.

The Caesar cipher involves replacing each letter in a message with the letter that is a certain number of letters after it in the alphabet. So, in an English message, we might replace each A with D, each B with E, each C with F, and so on, if shifting by three characters. We continue this approach all the way up to W, which is replaced with Z. Then, we let the substitution pattern ***wrap around***, so that we replace X with A, Y with B, and Z with C.

Converting Between Strings and Character Lists

Given that strings are immutable, we cannot directly edit an instance to encrypt it. Instead, our goal will be to generate a new string. A convenient technique for performing string transformations is to create an equivalent list of characters, edit the list, and then reassemble a (new) string based on the list. The first step can be performed by sending the string as a parameter to the constructor of the list class. For example, the expression list('bird') produces the result ['b', 'i', 'r', 'd']. Conversely, we can use a list of characters to build a string by invoking the join method on an empty string, with the list of characters as the parameter. For example, the call ''.join(['b', 'i', 'r', 'd']) returns the string 'bird'.

Using Characters as Array Indices

If we were to number our letters like array indices, so that A is 0, B is 1, C is 2, and so on, then we can write the Caesar cipher with a rotation of r as a simple formula: Replace each letter i with the letter $(i + r)$ mod 26, where mod is the ***modulo*** operator, which returns the remainder after performing an integer division. This operator is denoted with % in Python, and it is exactly the operator we need to easily perform the wrap around at the end of the alphabet. For 26 mod 26 is 0, 27 mod 26 is 1, and 28 mod 26 is 2. The decryption algorithm for the Caesar cipher is just the opposite—we replace each letter with the one r places before it, with wrap around (that is, letter i is replaced by letter $(i - r)$ mod 26).

We can represent a replacement rule using another string to describe the translation. As a concrete example, suppose we are using a Caesar cipher with a three-character rotation. We can precompute a string that represents the replacements that should be used for each character from A to Z. For example, A should be replaced by D, B replaced by E, and so on. The 26 replacement characters in order are `'DEFGHIJKLMNOPQRSTUVWXYZABC'`. We can subsequently use this translation string as a guide to encrypt a message. The remaining challenge is how to quickly locate the replacement for each character of the original message.

Fortunately, we can rely on the fact that characters are represented in Unicode by integer code points, and the code points for the uppercase letters of the Latin alphabet are consecutive (for simplicity, we restrict our encryption to uppercase letters). Python supports functions that convert between integer code points and one-character strings. Specifically, the function ord(c) takes a one-character string as a parameter and returns the integer code point for that character. Conversely, the function chr(j) takes an integer and returns its associated one-character string.

In order to find a replacement for a character in our Caesar cipher, we need to map the characters `'A'` to `'Z'` to the respective numbers 0 to 25. The formula for doing that conversion is j = ord(c) − ord(`'A'`). As a sanity check, if character c is `'A'`, we have that $j = 0$. When c is `'B'`, we will find that its ordinal value is precisely one more than that for `'A'`, so their difference is 1. In general, the integer j that results from such a calculation can be used as an index into our precomputed translation string, as illustrated in Figure 5.21.

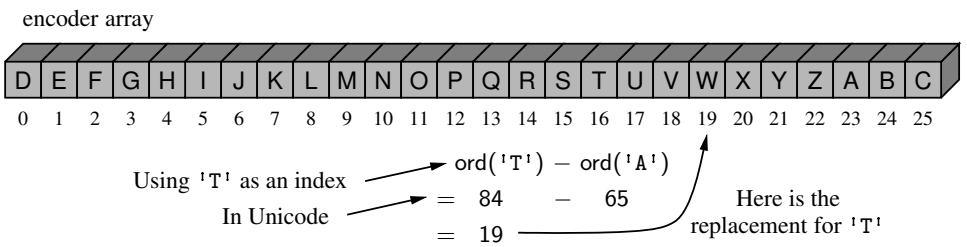


Figure 5.21: Illustrating the use of uppercase characters as indices, in this case to perform the replacement rule for Caesar cipher encryption.

In Code Fragment 5.11, we develop a Python class for performing the Caesar cipher with an arbitrary rotational shift, and demonstrate its use. When we run this program (to perform a simple test), we get the following output.

```
Secret:  WKH HDJOH LV LQ SODB; PHHW DW MRH'V.
Message: THE EAGLE IS IN PLAY; MEET AT JOE'S.
```

The constructor for the class builds the forward and backward translation strings for the given rotation. With those in hand, the encryption and decryption algorithms are essentially the same, and so we perform both by means of a nonpublic utility method named _transform.

```
class CaesarCipher:
  """Class for doing encryption and decryption using a Caesar cipher."""

  def __init__(self, shift):
    """Construct Caesar cipher using given integer shift for rotation."""
    encoder = [None] * 26                       # temp array for encryption
    decoder = [None] * 26                       # temp array for decryption
    for k in range(26):
      encoder[k] = chr((k + shift) % 26 + ord('A'))
      decoder[k] = chr((k - shift) % 26 + ord('A'))
    self._forward = ''.join(encoder)            # will store as string
    self._backward = ''.join(decoder)           # since fixed

  def encrypt(self, message):
    """Return string representing encripted message."""
    return  self._transform(message, self._forward)

  def decrypt(self, secret):
    """Return decrypted message given encrypted secret."""
    return  self._transform(secret, self._backward)

  def _transform(self, original, code):
    """Utility to perform transformation based on given code string."""
    msg = list(original)
    for k in range(len(msg)):
      if msg[k].isupper( ):
        j = ord(msg[k]) - ord('A')              # index from 0 to 25
        msg[k] = code[j]                        # replace this character
    return ''.join(msg)

if __name__ == '__main__':
  cipher = CaesarCipher(3)
  message = "THE EAGLE IS IN PLAY; MEET AT JOE'S."
  coded = cipher.encrypt(message)
  print('Secret: ', coded)
  answer = cipher.decrypt(coded)
  print('Message:', answer)
```

Code Fragment 5.11: A complete Python class for the Caesar cipher.